DeceptiCloud

AI-Driven Cyber Deception System using Adaptive Honeypots in Cloud Environment

Ahmed Ali (22P-9318)

Shoaib Akhtar (22P-9448)

Waleed Khan Kayani (22P-9139)

FAST National University of Computer and Emerging Sciences



Introduction

- Rapid growth of cloud computing introduces new security challenges.
- A new solution as defensive strategy for Cloud Environments.
- Employs Machine Learning models to analyze attacker behavior and predict threat patterns.
- Incorporates Wazuh for log monitoring, threat detection, and customized configuration for deception analysis.
- Deception technology offers a proactive way to mislead attackers.
- Honeypots simulate real systems to observe and study attack behavior.
- Decepti-Cloud aims to enhance cloud security using AI-driven adaptive honeypots.
- Provides much closer to real-time threat intelligence and behavior-based detection in cloud environments.



Motivation

- The Rise of the Cloud Computing
- Increasingly complex cyber threats in the cloud environments.
- Existing defenses struggle to detect zero-day and stealth attacks.
- Undetected attacks cause severe data breaches and financial loss.
- Adaptive honeypots and AI may be the Solution.



Problem Statement

- Traditional tools fail against sophisticated attacks in containerized environments
- Static honeypots easily identified; dynamic ones lack live adaptation
- Poor behavioral adaptation reduces engagement and intelligence gathering
- No intelligent routing to separate legitimate from malicious traffic
- Missing automated learning for emerging attack patterns
- Inadequate scalable cloud deception systems



Proposed Solution

- AI-driven adaptive honeypots with real-time behavioral adaptation in Docker/Kubernetes environments
- Intelligent traffic routing system to differentiate and redirect malicious traffic from legitimate users
- ML-powered threat detection, classification, and attacker behavior profiling
- Automated continuous learning mechanism to analyze and adapt to emerging attack patterns
- Real-time threat intelligence dashboard for visualization and monitoring



Scope of the Project

- Focus on private cloud containerized platforms: Docker and Kubernetes
- Detection and handling of DDoS and SQL/NoSQL injection attacks
- Development of two ML models: traffic classification and adaptive honeypot behavior
- Deployment of identical real and fake websites with separate databases
- Automated traffic routing and integration with Wazuh for centralized logging
- Dynamic IP/port rotation with consistency management and automated model retraining
- Real-time dashboard for monitoring and threat intelligence visualization
- Testing and evaluation in controlled private cloud testbeds



Objectives

- Build AI-driven system routing malicious traffic to adaptive honeypots
- Design honeypots with continuous learning from live attacker behavior
- Deploy ML-based traffic classifier for accurate threat detection
- Implement containerized infrastructure with real and decoy websites
- Enable automated model retraining using collected attack data
- Develop dynamic IP/port rotation to prevent honeypot detection
- Create real-time dashboard for threat intelligence visualization
- Evaluate system performance through controlled testbed experiments



Implementation Plan (Methodology)

Phase 1: Research & Requirements

- Literature Review
- Technology Stack Selection

Phase 2: System Design

- Architectural Blueprint
- Honeypot Design

Phase 3: Development & Integration

- Honeypot Coding
- AI Model Training
- Cloud Deployment

Phase 4: Testing & Evaluation

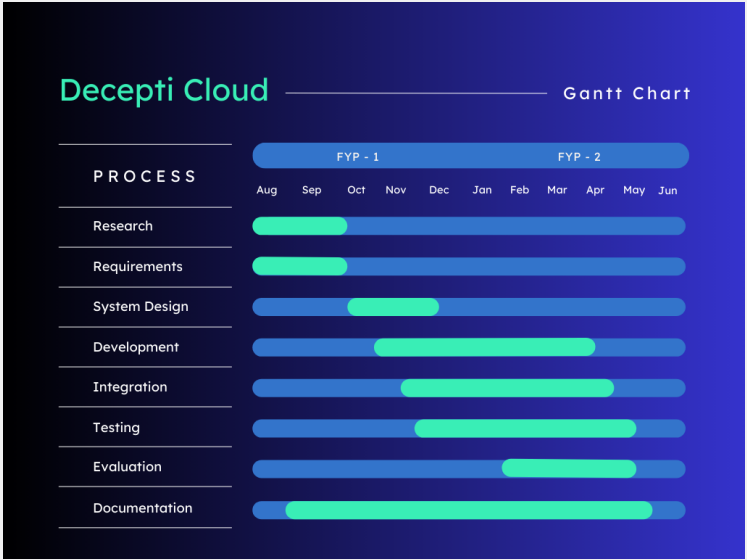
- Security Testing
- Performance Metrics

Phase 5: Documentation & Finalization

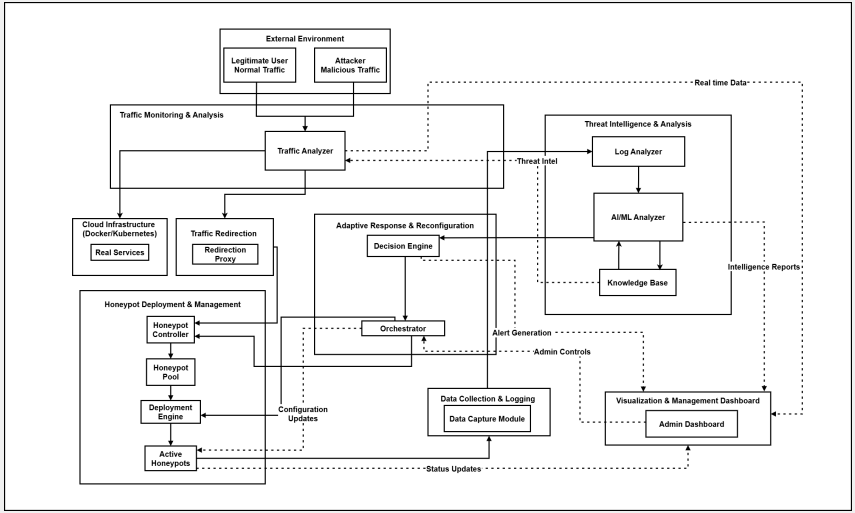
- Final Report
- Project Presentation



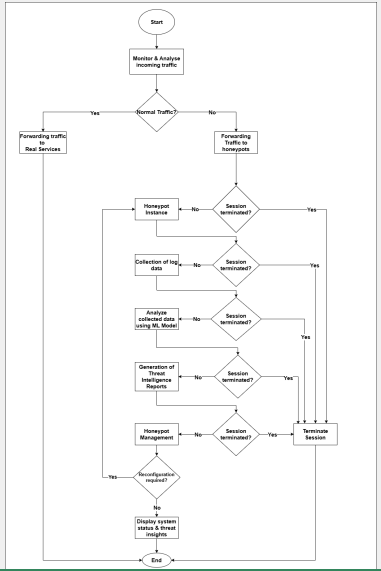
Timeline


Decepti Cloud
Gantt Chart
PROCESS
FYP - 1
FYP - 2
Aug
Sep
Oct
Nov
Dec
Jan
Feb
Mar
Apr
May
Jun
Research
Requirements
System Design
Development
Integration
Testing
Evaluation
Documentation

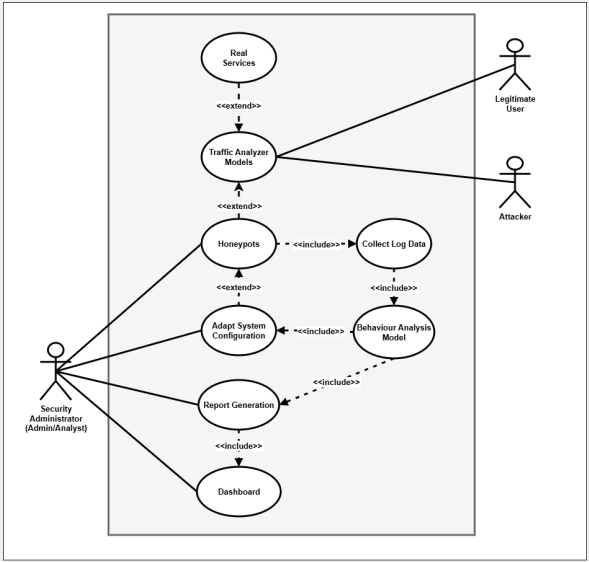
Architecture Diagram



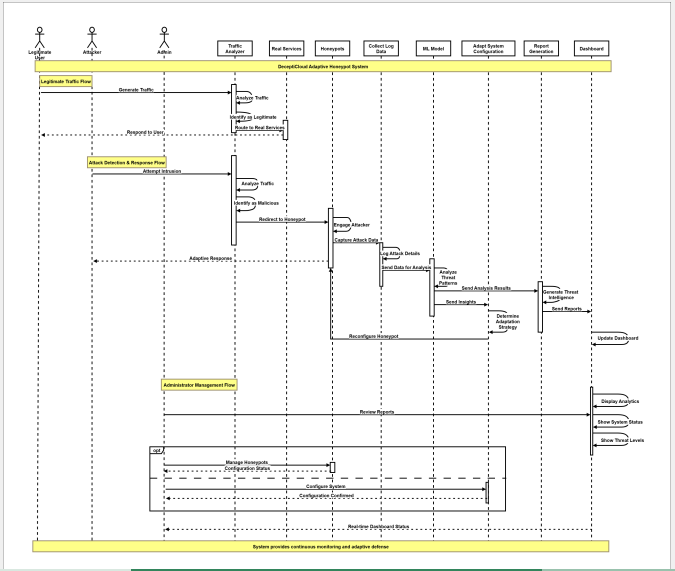
Flowchart Diagram



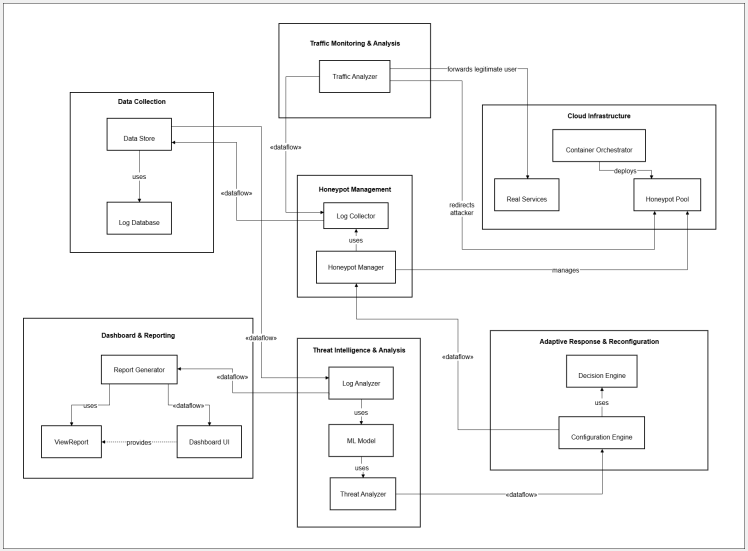
Use Case Diagram



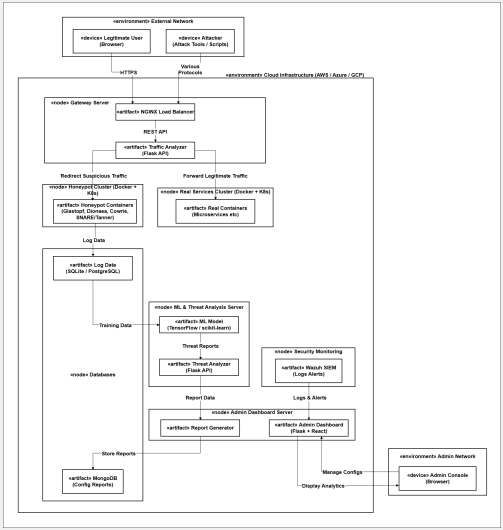
Sequence Diagram



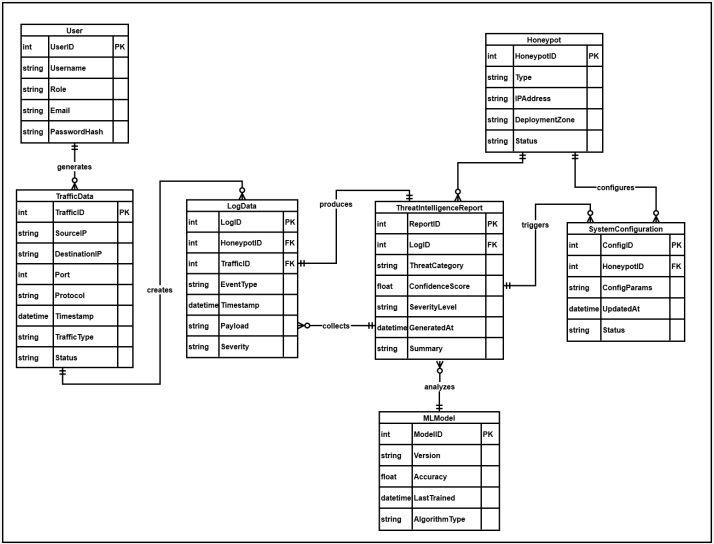
Component Diagram



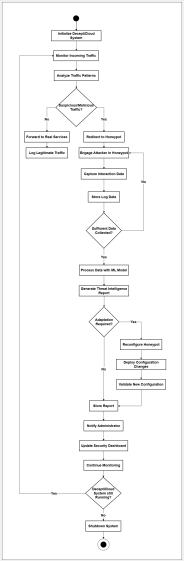
Deployment Diagram



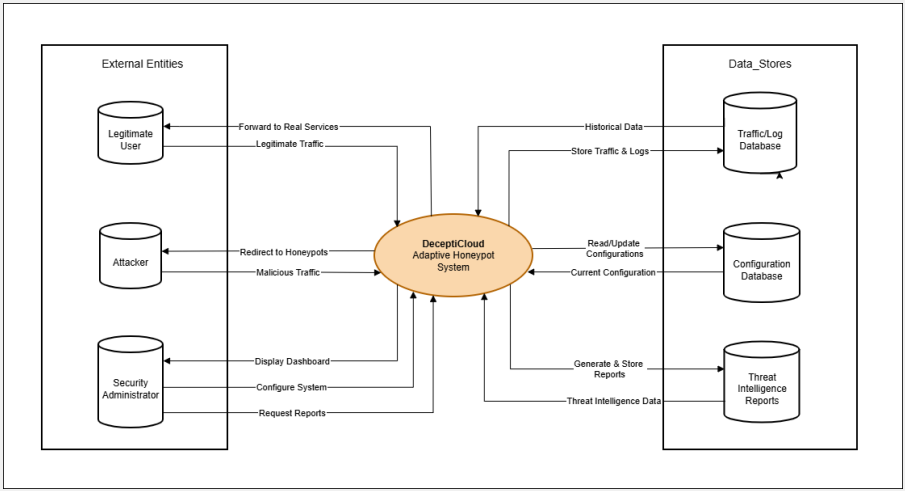
ER Diagram


User
int UserID PK
string Username
string Role
string Email
string PasswordHash
generates
TrafficData
int TrafficID PK
string SourceIP
string DestinationIP
int Port
string Protocol
datetime Timestamp
string TrafficType
string Status
creates
LogData
int LogID PK
int HoneypotID FK
int TrafficID FK
string EventType
datetime Timestamp
string Payload
string Severity
produces
collects
Honeypot
int HoneypotID PK
string Type
string IPAddress
string DeploymentZone
string Status
configures
ThreatIntelligenceReport
int ReportID PK
int LogID FK
string ThreatCategory
float ConfidenceScore
string SeverityLevel
datetime GeneratedAt
string Summary
triggers
SystemConfiguration
int ConfigID PK
int HoneypotID FK
string ConfigParams
datetime UpdatedAt
string Status
analyzes
MLModel
int ModelID PK
string Version
float Accuracy
datetime LastTrained
string AlgorithmType

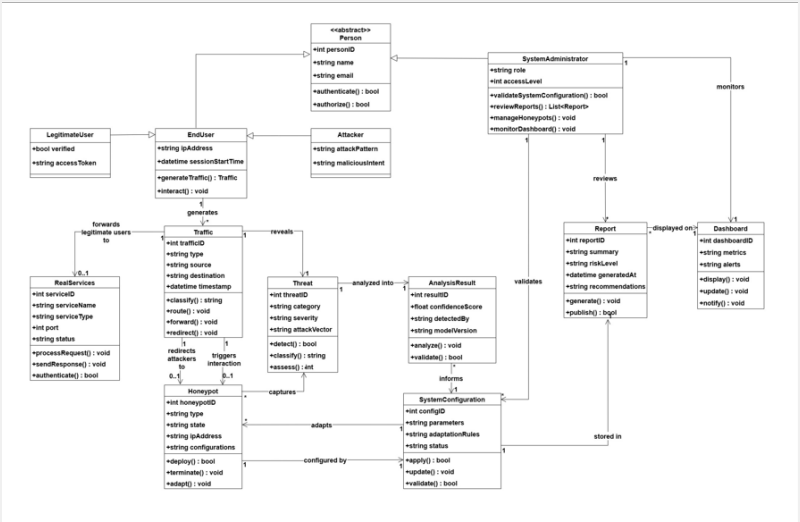
Activity Diagram



Context Diagram



Class Diagram



Literature Review

#	Title	Yr	Key Findings	Strengths	Limitations
1	Honeypot Implementation in Cloud	2023	Detected attacks, validated multi-cloud risks	Practical deployment, security improvements	Static, no ML/adaptive features
2	Container Orchestration Honeypots	2023	Captured crypto-jacking, attacker TTPs	Real world data, deep analysis	No adaptive response, static templates
3	HoneyChart: Automated Management over K8s	2023	Streamlined deployment, reduced errors	Automation, K8s integration	No adaptive features, manual updates
4	Honeypot Allocation Under Uncertainty	2022	Dynamic allocation, theoretical foundation	Rigorous modeling, uncertainty handling	Simulation only, no real deployment
5	Advancing with Honeypots and Deception	2025	Validated adaptive honeypots, ML benefits	Comprehensive analysis	No detailed implementation
6	TRAPNET AI: AI-Driven Deception	2025	Real-time detection, multi-vector coverage	Practical, user-friendly	Limited scope, no adaptive reconfig
7	AI-Driven Analysis of Honeypot Data	2024	Faster detection, improved intelligence	Novel LLM use, practical details	High cost, no adaptive honeypots
8	Cloud Computing: AI-Driven Mitigation	2024	Identified risks, AI-based protection	Broad analysis	Lacks depth, no honeypot focus



Literature Review - Comparison Table

Reference	Cloud Deploy	K8's Native	Higher Interact	Dynamic	Adaptive	AI/ML Detect	Auto Response	RL/Game Theory	LLM/AI Analytics	LAB Prod
Honeypot Implementation	✓		✓							✓
Container Orchestration	✓	✓	✓							✓
HoneyChart	✓	✓	✓							✓
Honeypot Allocation					✓	✓		✓		
Advancing Cybersecurity	✓		✓		✓	✓		✓		✓
TRAPNET AI			✓			✓	✓			✓
Intelligent Threat Detection						✓			✓	✓
Multi-Cloud Security	✓						✓			
Decepti-Cloud	✓	✓	✓	✓	✓	✓	✓			✓



Test Cases

Test Case ID: TC-DC-01
Test Scenario: Deploy DeceptiCloud system with Docker containers and Kubernetes orchestration
Pre Conditions: Kubernetes cluster is configured and accessible. Docker images for real and fake websites are available. Database schemas are defined.
Post Conditions: DeceptiCloud system is operational. Both real and fake infrastructures are running. System ready to route traffic.

Test Case ID: TC-DC-02
Test Scenario: Detect SQL/NoSQL injection and DDoS attacks using ML traffic analyzer
Preconditions: ML Model 1 is trained and deployed. Traffic monitoring is active. Baseline traffic patterns are established.
Post Conditions: Traffic classification completed. Decision made to route to real or fake website. Detection information sent to dashboard.



Test Cases Continue

Test Case ID: TC-DC-03
Test Scenario: Redirect attacker traffic to fake website infrastructure
Pre Conditions: ML Model 1 has classified traffic as malicious. Fake website is operational. Routing rules are configured.
Post Conditions: Fake website served attacker. Real infrastructure unaffected. Attack data collected for analysis and visible on dashboard. Honeypot engagement recorded.

Test Case ID: TC-DC-04
Test Scenario: Simulate website crash on fake infrastructure during DDoS attack
Preconditions: DDoS attack detected by ML Model 1. Attacker traffic routed to fake website. Crash simulation parameters configured.
Post-conditions: Fake website in simulated crashed state. Attacker satisfied with DDoS success. Real service uninterrupted. Attack pattern stored for Behaviour Analysis Model training. Dashboard shows incident report.



Test Cases Continue

Test Case ID: TC-DC-05
Test Scenario: Serve fabricated database records during injection attack
Pre Conditions: Injection attack detected by ML Model 1. Attacker routed to fake website. Fake database populated with fabricated records.
Post Conditions: Fake data delivered to attacker. Injection technique captured and displayed on dashboard. Attacker profiled for future detection. Real data protected.

Test Case ID: TC-DC-06
Test Scenario: Continuously change honeypot IPs and ports while maintaining consistency for concurrent attackers
Pre Conditions: Behaviour Analysis Model is operational. IP/port pool is available. Attack sessions are being tracked. Network configuration allows dynamic binding. Dashboard is monitoring honeypot status.
Post Conditions: New IP/port active for honeypot. Attacker session tracking updated. Configuration change logged and visible on dashboard. Network routes updated.



Test Cases Continue

Test Case ID: TC-DC-07
Test Scenario: Multiple coordinated attackers see identical honeypot configuration
Preconditions: Multiple attackers accessing system within same time window. Behaviour Analysis Model tracking attacker sessions. Session correlation enabled. Dashboard monitoring concurrent sessions.
Post-conditions: Consistency group maintained. Attackers receive synchronized view. Session correlation logged and displayed on dashboard. Deception integrity preserved.

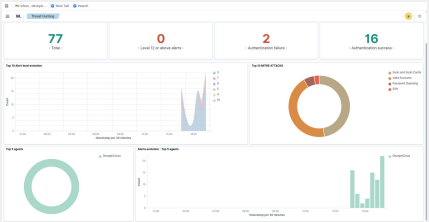
Test Case ID: TC-DC-08
Test Scenario: Integrate Wazuh with dashboard for centralized monitoring and real-time alert visualization
Preconditions: Wazuh is installed and configured. Honeypots are sending logs to Wazuh. Dashboard application is running and connected to Wazuh API.
Post Conditions: Dashboard displays complete security posture. Wazuh integration active. Security team can monitor all honeypot activities from centralized dashboard.

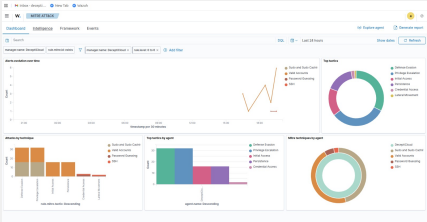
Test Cases Continue

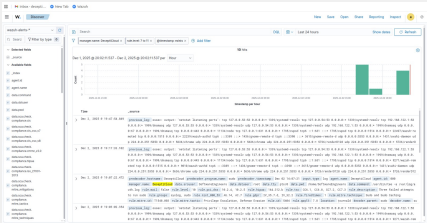
Test Case ID: TC-DC-09
Test Scenario: Automatically retrain Behaviour Analysis Model when novel attack patterns detected
Pre conditions: New attack pattern logged in system. Wazuh has captured attack data. Dashboard displays the new attack type. Labeled training data accumulated.
Post Conditions: Updated Behaviour Analysis Model deployed. Detection capabilities enhanced. Training completion logged and visible on dashboard.

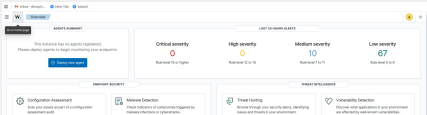
Test Case ID: TC-DC-10
Test Scenario: Security analyst accesses comprehensive reports and visualizations from integrated dashboard
Pre Conditions: Dashboard is operational with Wazuh integration. Historical data exists in system. Active honeypot sessions running. User authenticated with appropriate permissions.
Post Conditions: Analyst has comprehensive view of security status. Search history saved. Export file downloaded. Dashboard session logged.

Installation and Configuration of Wazuh

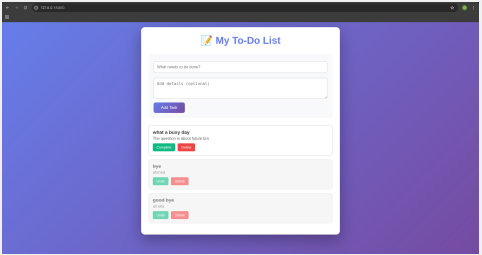


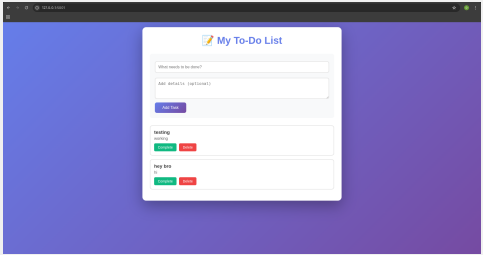


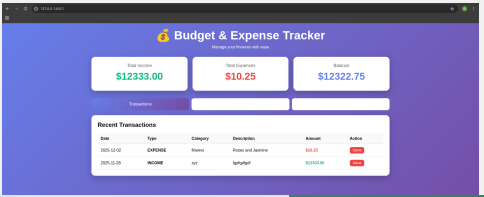


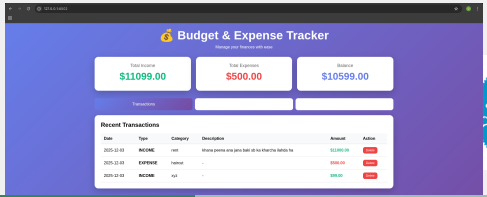


Real Services Fake Services









T-Pot Honeypot

```
PLAY [T-Pot - Install service] *************************************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************
ok: [127.0.0.1]

TASK [Install systemd service (All)] *******************************************************************************************************************
ok: [127.0.0.1]

PLAY [T-Pot - Setup a randomized daily reboot] **********************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************************************************
ok: [127.0.0.1]

TASK [Setup a randomized daily reboot (All)] ************************************************************************************************************
changed: [127.0.0.1]

PLAY RECAP *********************************************************************************************************************************************
127.0.0.1                  : ok=41   changed=9    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

### Playbook was successful.


### Choose your T-Pot type:
### (M)ive   - T-Pot Standard / HIVE installation.
###            Includes also everything you need for a distributed setup with sensors.
### (S)ensor - T-Pot Sensor installation.
###            Optimized for a distributed installation, without WebUI, Elasticsearch and Kibana.
### (L)LM    - T-Pot LLM installation.
###            Uses LLM based honeypots Beelzebub & Galah.
###            Requires Ollama (recommended) or ChatGPT subscription.
### M(i)ni   - T-Pot Mini installation.
###            Run 30+ honeypots with just a couple of honeypot daemons.
### (M)obile - T-Pot Mobile installation.
###            Includes everything to run T-Pot Mobile (available separately).
### (T)arpit - T-Pot Tarpit installation.
###            Feed data endlessly to attackers, bots and scanners.
###            Also runs a Denial of Service Honeypot (ddospot).

### Install Type? (h/s/l/i/m/t) h

### Installing T-Pot Standard / HIVE.

### T-Pot User Configuration ...
```



ML Model Challenges

```
All models saved in 'sqli_models/' folder

============================================================
OVERALL BEST MODELS
============================================================
modified_sql        : XGBoost         (F1: 0.9969, Acc: 0.9969)
sql_queries         : RandomForest    (F1: 0.9957, Acc: 0.9965)
sqli_extended       : XGBoost         (F1: 0.9931, Acc: 0.9931)
```

```
============================================================
OVERALL BEST MODELS
============================================================
nosql_40k           : RandomForest    (F1: 1.0000, Acc: 1.0000)
nosql_payloads      : RandomForest    (F1: 1.0000, Acc: 1.0000)
```

```
Training SVM model...

================================================
     RESULTS
================================================
Accuracy: 0.9917
F1 Score: 0.9917

Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      0.99      3908
           1       1.00      0.99      0.99      3907

    accuracy                           0.99      7815
   macro avg       0.99      0.99      0.99      7815
weighted avg       0.99      0.99      0.99      7815
```



ML Model Challenges Cont.

```
============================================================
DDoS PREPROCESSING COMPLETE! ✓
============================================================

Files saved:
- balanced_splits.pkl, balanced_scaler.pkl, balanced_label_encoder.pkl
- unbalanced_splits.pkl, unbalanced_scaler.pkl, unbalanced_label_encoder.pkl
- network_traffic_splits.pkl, network_traffic_scaler.pkl, network_traffic_label_encoder.pkl
```

```
============================================================
OVERALL BEST MODELS SUMMARY
============================================================
BALANCED            → DecisionTree    (F1: 1.0000, Acc: 1.0000)
UNBALANCED          → XGBoost         (F1: 1.0000, Acc: 1.0000)
NETWORK_TRAFFIC     → SVM             (F1: 1.0000, Acc: 1.0000)
```

	Accuracy	Precision	Recall	F1 Score
Random Forest	63.31	46.57	98.60	63.26
LightGBM	73.94	55.28	97.76	70.62
XGBoost	75.13	56.44	98.02	71.64
Ensemble	69.50	51.25	98.39	67.40

ML Model Result

```
================================================================================
FINAL MODEL COMPARISON (TEST SET)
================================================================================
               Accuracy  Precision  Recall  F1 Score
Random Forest     63.17      46.48   98.85     63.23
LightGBM          70.32      51.96   97.88     67.88
XGBoost           77.34      58.77   98.07     73.50
Ensemble          82.47      65.29   96.65     77.94

 Best Model: Ensemble (Test F1: 77.94%)

================================================================================
PIPELINE COMPLETED SUCCESSFULLY
================================================================================
```

```
Predicting with Random Forest...
Random Forest Results:
   Accuracy:  93.15%
   Precision: 99.49%
   Recall:    60.03%
   F1-Score:  74.88%

Predicting with XGBoost...
XGBoost Results:
   Accuracy:  93.80%
   Precision: 95.84%
   Recall:    66.40%
   F1-Score:  78.45%

Predicting with LightGBM...
LightGBM Results:
   Accuracy:  94.19%
   Precision: 99.12%
   Recall:    66.38%
   F1-Score:  79.51%

Predicting with Voting Ensemble...

Ensemble Results:
   Accuracy:  94.21%
   Precision: 99.26%
   Recall:    66.45%
   F1-Score:  79.60%
```

THANK YOU!

Time for Q&A

